AI-ASSISTED CODING

ASSIGNMENT-11.5

NAME :V.SIDDARTHA
HALL:2303A52062
BATCH:45

ask Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
Sample Input Code:
class Stack:
pass
Expected Output:
• A functional stack implementation with all required methods and Docstrings
PROMPT:
Map Smart E-Commerce features to best data structures with justification and implement one selected feature as runnable Python code.

CODE:

```
from collections import deque

def print_ecommerce_mapping():
    rows = [
        (
            "Shopping Cart Management",
            "Linked List",
            "Cart items change dynamically with frequent insert/remove actions. "
            "Linked list is suitable for flexible structural updates.",
        ),
        (
            "Order Processing System",
            "Queue",
            "Orders should be handled in the sequence they are placed. "
            "Queue ensures FIFO processing and fair order fulfillment.",
        ),
        (
            "Top-Selling Products Tracker",
            "Priority Queue",
            "Products must be ranked continuously by sales counts. "
            "Priority queue helps retrieve highest-selling products quickly.",
```

```
        ),
        (
            "Product Search Engine",
            "Hash Table",
            "Product ID based search must be fast and direct. "
            "Hash table gives near O(1) lookup for product records.",
        ),
        (
            "Delivery Route Planning",
            "Graph",
            "Warehouses and delivery points are connected locations. "
            "Graph supports route optimization and path traversal.",
        ),
    ]

    print("Task 8 Output")
    print("Feature | Data Structure | Justification")
    print("-" * 95)
    for feature, ds, why in rows:
        print(f"{feature} | {ds} | {why}")

class OrderQueue:
    """FIFO order processing queue for e-commerce orders."""

    def __init__(self):
        self._orders = deque()

    def place_order(self, order_id):
        """Add a new order to the processing queue."""
        self._orders.append(order_id)

    def process_order(self):
        """Process next order in queue."""
        if not self._orders:
            return None
        return self._orders.popleft()

if __name__ == "__main__":
    print_ecommerce_mapping()
    oq = OrderQueue()
    oq.place_order("ORD1001")
```

```
oq.place_order("ORD1002")
oq.place_order("ORD1003")
print("\nImplemented Feature Demo: Order Processing System")
print(oq.process_order())
print(oq.process_order())
```

OUTPUT:

```
13        ),

PROBLEMS   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK8.PY"
Task 8 Output
Feature | Data Structure | Justification
-----------------------------------------------------------------------------------------------
Shopping Cart Management | Linked List | Cart items change dynamically with frequent insert/remove actions. Linked list is suitab
le for flexible structural updates.
Order Processing System | Queue | Orders should be handled in the sequence they are placed. Queue ensures FIFO processing and fai
r order fulfillment.
Top-Selling Products Tracker | Priority Queue | Products must be ranked continuously by sales counts. Priority queue helps retrie
ve highest-selling products quickly.
Product Search Engine | Hash Table | Product ID based search must be fast and direct. Hash table gives near O(1) lookup for produ
ct records.
Delivery Route Planning | Graph | Warehouses and delivery points are connected locations. Graph supports route optimization and p
ath traversal.

Implemented Feature Demo: Order Processing System
ORD1001
ORD1002
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>

powershell
Python
Python
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- **FIFO-based queue class with enqueue, dequeue, peek, and size methods.**

PROMPT:

Implement a Queue class using Python lists with enqueue, dequeue, peek, and size methods

CODE:

```
class Queue:
    """FIFO queue implementation using a Python list."""

    def __init__(self):
        self._items = []

    def enqueue(self, value):
```

```
        """Add an element to the back of the queue."""
        self._items.append(value)

    def dequeue(self):
        """Remove and return the front element.

        Raises:
            IndexError: If queue is empty.
        """
        if not self._items:
            raise IndexError("Cannot dequeue from an empty queue")
        return self._items.pop(0)

    def peek(self):
        """Return front element without removing it."""
        if not self._items:
            raise IndexError("Cannot peek an empty queue")
        return self._items[0]

    def size(self):
        """Return number of elements in queue."""
        return len(self._items)

if __name__ == "__main__":
    q = Queue()
    q.enqueue("A")
    q.enqueue("B")
    print("Task 2 Output")
    print("Peek:", q.peek())
    print("Dequeue:", q.dequeue())
    print("Size:", q.size())
```

OUTPUT:

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK2.PY"
Task 2 Output
Peek: A
Dequeue: A
Size: 1
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

PROMPT

Implement a singly linked list with insert and display methods and clear method documentation.

CODE

```
class Node:
    """Single node for linked list."""

    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """Singly linked list implementation."""

    def __init__(self):
        self.head = None

    def insert(self, data):
```

```
        """Insert a new node at the end of the linked list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return

        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    def display(self):
        """Return all node values as a list."""
        values = []
        current = self.head
        while current is not None:
            values.append(current.data)
            current = current.next
        return values

if __name__ == "__main__":
    ll = LinkedList()
    ll.insert(1)
    ll.insert(2)
    ll.insert(3)
    print("Task 3 Output")
    print("Linked List:", ll.display())
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK3.PY"
Task 3 Output
Linked List: [1, 2, 3]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

PROMPT:

Implement a hash table with insert, search, and delete methods using chaining for collision handling.

CODE:

```
class HashTable:
    """Simple hash table using separate chaining with lists."""

    def __init__(self, capacity=10):
        self.capacity = capacity
        self.table = [[] for _ in range(capacity)]

    def _index(self, key):
        """Compute bucket index for a key."""
        return hash(key) % self.capacity

    def insert(self, key, value):
        """Insert or update key-value pair."""
        idx = self._index(key)
        bucket = self.table[idx]

        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return

        bucket.append((key, value))

    def search(self, key):
        """Return value for key, or None if not found."""
        idx = self._index(key)
        for k, v in self.table[idx]:
            if k == key:
                return v
        return None

    def delete(self, key):
```

```
        """Delete key from table and return True if deleted."""
        idx = self._index(key)
        bucket = self.table[idx]

        for i, (k, _) in enumerate(bucket):
            if k == key:
                del bucket[i]
                return True
        return False

if __name__ == "__main__":
    ht = HashTable()
    ht.insert("id1", "Alice")
    ht.insert("id2", "Bob")
    print("Task 4 Output")
    print("Search id1:", ht.search("id1"))
    print("Delete id2:", ht.delete("id2"))
    print("Search id2:", ht.search("id2"))
```

OUTPUT:

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK4.PY"
Task 4 Output
Search id1: Alice
Delete id2: True
Search id2: None
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- **Graph with methods to add vertices, add edges, and display connections.**

PROMPT

Implement a graph using adjacency list with methods to add vertices, add edges, and display connections

CODE

```
class Graph:
    """Undirected graph represented with adjacency list."""

    def __init__(self):
        self.adj = {}

    def add_vertex(self, vertex):
        """Add vertex if it does not exist."""
        if vertex not in self.adj:
            self.adj[vertex] = []

    def add_edge(self, u, v):
        """Add an undirected edge between u and v."""
        self.add_vertex(u)
        self.add_vertex(v)
        if v not in self.adj[u]:
            self.adj[u].append(v)
        if u not in self.adj[v]:
            self.adj[v].append(u)

    def display(self):
        """Return adjacency mapping."""
        return self.adj

if __name__ == "__main__":
    g = Graph()
    g.add_edge("WardA", "WardB")
    g.add_edge("WardB", "WardC")
    print("Task 5 Output")
    print("Connections:", g.display())
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK5.PY"
Task 5 Output
Connections: {'WardA': ['WardB'], 'WardB': ['WardA', 'WardC'], 'WardC': ['WardB']}
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

ask Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT

Map Smart Hospital features to best data structures with justification and implement one selected feature as runnable Python code.

CODE

```
from collections import deque
import heapq

def print_hospital_mapping():
    rows = [
        (
            "Patient Check-In System",
            "Queue",
            "Patients are processed in arrival order, which matches FIFO behavior. "
            "A queue keeps insertion and removal straightforward and fair.",
        ),
        (
            "Emergency Case Handling",
            "Priority Queue",
            "Critical patients must bypass normal order. "
            "Priority queue gives faster treatment based on severity value.",
        ),
        (
            "Medical Records Storage",
            "Hash Table",
            "Records are accessed by patient ID. "
            "Hash table enables near O(1) lookup, insert, and update for IDs.",
        ),
        (
            "Doctor Appointment Scheduling",
            "Binary Search Tree (BST)",
            "Appointments are naturally sorted by time. "
            "BST supports ordered traversal and range-based time queries.",
        ),
        (
            "Hospital Room Navigation",
            "Graph",
            "Rooms and wards are connected like nodes and edges. "
            "Graph supports path discovery and navigation across connections.",
        ),
    ]

    print("Task 6 Output")
    print("Feature | Data Structure | Justification")
    print("-" * 95)
    for feature, ds, why in rows:
```

```
        print(f"{feature} | {ds} | {why}")

class EmergencyQueue:
    """Priority queue for emergency patient handling.

    Lower priority number means higher urgency.
    """

    def __init__(self):
        self._heap = []
        self._seq = 0

    def add_patient(self, patient_name, priority):
        """Add patient with priority to emergency queue."""
        heapq.heappush(self._heap, (priority, self._seq, patient_name))
        self._seq += 1

    def treat_next(self):
        """Return next highest-priority patient."""
        if not self._heap:
            return None
        priority, _, name = heapq.heappop(self._heap)
        return {"patient": name, "priority": priority}

if __name__ == "__main__":
    print_hospital_mapping()
    er = EmergencyQueue()
    er.add_patient("Ravi", 3)
    er.add_patient("Anita", 1)
    er.add_patient("Kiran", 2)
    print("\nImplemented Feature Demo: Emergency Case Handling")
    print(er.treat_next())
    print(er.treat_next())
```

OUTPUT

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK6.PY"
Task 6 Output
Feature | Data Structure | Justification
-----------------------------------------------------------------------------------------------
Patient Check-In System | Queue | Patients are processed in arrival order, which matches FIFO behavior. A queue keeps insertion a
nd removal straightforward and fair.
Emergency Case Handling | Priority Queue | Critical patients must bypass normal order. Priority queue gives faster treatment base
d on severity value.
Medical Records Storage | Hash Table | Records are accessed by patient ID. Hash table enables near O(1) lookup, insert, and updat
e for IDs.
Doctor Appointment Scheduling | Binary Search Tree (BST) | Appointments are naturally sorted by time. BST supports ordered traver
sal and range-based time queries.
Hospital Room Navigation | Graph | Rooms and wards are connected like nodes and edges. Graph supports path discovery and navigati
on across connections.

Implemented Feature Demo: Emergency Case Handling
{'patient': 'Anita', 'priority': 1}
{'patient': 'Kiran', 'priority': 2}
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

ask Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:
Map Smart City Traffic features to best data structures with justification and implement one selected feature as runnable Python code.

CODE:

```
import heapq

def print_traffic_mapping():
    rows = [
        (
            "Traffic Signal Queue",
            "Queue",
            "Vehicles wait and move in arrival order at a signal. "
            "Queue models FIFO flow and avoids unfair overtaking.",
        ),
        (
            "Emergency Vehicle Priority Handling",
            "Priority Queue",
            "Emergency vehicles need faster movement than regular traffic. "
            "Priority queue places ambulances/fire trucks ahead by urgency.",
        ),
        (
            "Vehicle Registration Lookup",
            "Hash Table",
            "Lookup by registration number must be instant. "
            "Hash tables provide very fast key-based search and updates.",
        ),
        (
            "Road Network Mapping",
            "Graph",
            "Intersections and roads form connected nodes and edges. "
            "Graphs support route planning and connectivity analysis.",
        ),
        (
            "Parking Slot Availability",
            "Deque",
            "Slots can be assigned and released from both ends in peak scenarios. "
            "Deque supports efficient operations on both front and rear.",
        ),
    ]
```

```
    print("Task 7 Output")
    print("Feature | Data Structure | Justification")
    print("-" * 95)
    for feature, ds, why in rows:
        print(f"{feature} | {ds} | {why}")

class TrafficPriorityLane:
    """Priority handling lane for vehicles at a traffic control point."""

    def __init__(self):
        self._heap = []
        self._counter = 0

    def add_vehicle(self, plate_number, priority):
        """Add vehicle with priority (1 is highest)."""
        heapq.heappush(self._heap, (priority, self._counter, plate_number))
        self._counter += 1

    def dispatch(self):
        """Dispatch next vehicle based on priority."""
        if not self._heap:
            return None
        priority, _, plate = heapq.heappop(self._heap)
        return {"vehicle": plate, "priority": priority}

if __name__ == "__main__":
    print_traffic_mapping()
    lane = TrafficPriorityLane()
    lane.add_vehicle("TS09AB1111", 3)
    lane.add_vehicle("TS09EM0001", 1)
    lane.add_vehicle("TS09AB2222", 4)
    print("\nImplemented Feature Demo: Emergency Vehicle Priority Handling")
    print(lane.dispatch())
    print(lane.dispatch())
```

OUTPUT

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK6.PY"
Task 6 Output
Feature | Data Structure | Justification
-----------------------------------------------------------------------------------------------
Patient Check-In System | Queue | Patients are processed in arrival order, which matches FIFO behavior. A queue keeps insertion a
nd removal straightforward and fair.
Emergency Case Handling | Priority Queue | Critical patients must bypass normal order. Priority queue gives faster treatment base
d on severity value.
Medical Records Storage | Hash Table | Records are accessed by patient ID. Hash table enables near O(1) lookup, insert, and updat
e for IDs.
Doctor Appointment Scheduling | Binary Search Tree (BST) | Appointments are naturally sorted by time. BST supports ordered traver
sal and range-based time queries.
Hospital Room Navigation | Graph | Rooms and wards are connected like nodes and edges. Graph supports path discovery and navigati
on across connections.

Implemented Feature Demo: Emergency Case Handling
{'patient': 'Anita', 'priority': 1}
{'patient': 'Kiran', 'priority': 2}
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- **For each feature, select the most appropriate data structure from the list below:**
  - **o Stack**
  - **o Queue**
  - **o Priority Queue**
  - **o Linked List**
  - **o Binary Search Tree (BST)**
  - **o Graph**
  - **o Hash Table**
  - **o Deque**
- **Justify your choice in 2–3 sentences per feature.**
- **Implement one selected feature as a working Python program with AI-assisted code generation.**

Expected Output:

- **A table mapping feature → chosen data structure → justification.**

• A functional Python program implementing the chosen feature
with comments and docstrings.

PROMPT
Map Smart E-Commerce features to best data structures with justification and implement one selected feature as runnable Python code.

CODE

```
from collections import deque

def print_ecommerce_mapping():
    rows = [
        (
            "Shopping Cart Management",
            "Linked List",
            "Cart items change dynamically with frequent insert/remove actions. "
            "Linked list is suitable for flexible structural updates.",
        ),
        (
            "Order Processing System",
            "Queue",
            "Orders should be handled in the sequence they are placed. "
            "Queue ensures FIFO processing and fair order fulfillment.",
        ),
        (
            "Top-Selling Products Tracker",
            "Priority Queue",
            "Products must be ranked continuously by sales counts. "
            "Priority queue helps retrieve highest-selling products quickly.",
        ),
        (
            "Product Search Engine",
            "Hash Table",
            "Product ID based search must be fast and direct. "
            "Hash table gives near O(1) lookup for product records.",
        ),
        (
            "Delivery Route Planning",
            "Graph",
            "Warehouses and delivery points are connected locations. "
```

```
        "Graph supports route optimization and path traversal.",
    ),
]

print("Task 8 Output")
print("Feature | Data Structure | Justification")
print("-" * 95)
for feature, ds, why in rows:
    print(f"{feature} | {ds} | {why}")

class OrderQueue:
    """FIFO order processing queue for e-commerce orders."""

    def __init__(self):
        self._orders = deque()

    def place_order(self, order_id):
        """Add a new order to the processing queue."""
        self._orders.append(order_id)

    def process_order(self):
        """Process next order in queue."""
        if not self._orders:
            return None
        return self._orders.popleft()

if __name__ == "__main__":
    print_ecommerce_mapping()
    oq = OrderQueue()
    oq.place_order("ORD1001")
    oq.place_order("ORD1002")
    oq.place_order("ORD1003")
    print("\nImplemented Feature Demo: Order Processing System")
    print(oq.process_order())
    print(oq.process_order())
```

OUTPUT

PROBLEMS DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

powershell
Python
Python

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/11.5.PY/TASK8.PY"
Task 8 Output
Feature | Data Structure | Justification
-----------------------------------------------------------------------------------------------
Shopping Cart Management | Linked List | Cart items change dynamically with frequent insert/remove actions. Linked list is suitab
le for flexible structural updates.
Order Processing System | Queue | Orders should be handled in the sequence they are placed. Queue ensures FIFO processing and fai
r order fulfillment.
Top-Selling Products Tracker | Priority Queue | Products must be ranked continuously by sales counts. Priority queue helps retrie
ve highest-selling products quickly.
Product Search Engine | Hash Table | Product ID based search must be fast and direct. Hash table gives near O(1) lookup for produ
ct records.
Delivery Route Planning | Graph | Warehouses and delivery points are connected locations. Graph supports route optimization and p
ath traversal.

Implemented Feature Demo: Order Processing System
ORD1001
ORD1002
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```